

# SOFTWARE CHANGE IMPLEMENTATION REPORT

## CR-004: Student Input Field Validation

| Field                 | Details                        |
|-----------------------|--------------------------------|
| Change Request ID     | CR-004                         |
| Change Request Title  | Student Input Field Validation |
| Student Name          | C. Lavasujan                   |
| Student ID            | 24ICT06                        |
| Implementation Status | Completed                      |
| Date                  | 09 October 2026                |

## 1. Change Request Summary

### 1.1 Business Requirement

Student Add and Edit operations must validate student information consistently before records are created or updated. The application must reject missing, malformed, insecure, or whitespace-padded values, show clear feedback in `StudentFormModal.jsx`, and enforce the same constraints on the backend so direct API requests cannot bypass client-side checks.

### 1.2 Existing Behaviour

The change strengthens and standardizes field validation across the student form and server-side controller. The supplied implementation report identifies the following areas requiring validation and protection:

- Required values must not pass validation when they are empty or contain only whitespace.
- First and last names require minimum-length and alphabetic-content checks.
- Email and phone values require explicit format checks.
- New student passwords require a strong-password policy.
- Direct API requests must be checked independently of frontend validation.

### 1.3 Requested Change

Apply consistent client-side and server-side validation to student Add and Edit operations. The field rules are:

- Registration Number is required and cannot be blank or whitespace-only after trimming.
- First Name is required, at least 3 characters long, and alphabetic only.
- Last Name is required, at least 3 characters long, and alphabetic only.
- Email is required and must match the configured valid email format.
- Phone Number is required, numeric only, and exactly 10 digits long.
- New-student passwords must be at least 8 characters and include uppercase, lowercase, numeric, and special characters.
- Trim string inputs before validating them; return HTTP 400 with a descriptive message for invalid backend requests.

- Preserve existing CRUD, authentication, authorization, password hashing, routing, and database access behaviour.

## 2. Implementation

### 2.1 Implementation Summary

Client-side validation was added to `StudentFormModal.jsx` and server-side validation was added to `studentController.js`. The frontend uses the existing `formError` state to show precise validation messages and prevents invalid form submission. The controller independently trims and validates request data in `addStudent` and `editStudent`, returning HTTP 400 before database work when a rule fails. The existing model and database schema were not redesigned.

### 2.2 Affected Files

| File / Component                                          | Change    | Purpose                                                                                                                                      |
|-----------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>frontend/src/components/StudentFormModal.jsx</code> | Modified  | Validates Add / Edit values, trims input, blocks invalid submission, and shows descriptive form errors.                                      |
| <code>backend/src/controllers/studentController.js</code> | Modified  | Independently validates and trims request fields in <code>addStudent</code> / <code>editStudent</code> ; rejects invalid data with HTTP 400. |
| <code>backend/src/models/studentModel.js</code>           | Unchanged | Existing SQL queries and data-access model are preserved.                                                                                    |
| Database schema                                           | Unchanged | No schema redesign or migration is required by CR-004.                                                                                       |
| Authentication / authorization middleware                 | Unchanged | Existing administrator access controls are preserved.                                                                                        |

### 2.3 Technical Changes

#### Client-Side Validation

- `StudentFormModal.jsx` checks required fields before issuing a request.
- First and last names are trimmed, checked for a minimum length of three characters, and restricted to alphabetic characters.
- Email syntax and the exact 10-digit numeric phone rule are checked before submission.
- New student passwords must satisfy the eight-character minimum and contain uppercase, lowercase, number, and special character requirements.
- The pre-existing `formError` UI state displays a user-readable validation message.

#### Server-Side Validation and Security

- `studentController.js` repeats the validation independently for Add and Edit endpoints, including requests that bypass the UI.
- Invalid requests are rejected with HTTP 400 and a descriptive JSON error before database operations.
- String inputs are trimmed on both frontend and backend to prevent whitespace padding from bypassing required-field checks.
- `studentModel.js`, student table schema, existing authentication / authorization middleware, and password hashing were preserved.

### 2.4 Implementation Logic

1. An administrator opens the Add Student or Edit Student form and enters student data.
2. The frontend trims string inputs and validates each field against the documented rules.

3. If a frontend rule fails, the form displays a specific error through `formError` and does not submit the request.
4. If client validation succeeds, the form sends the existing API request.
5. The backend controller trims and independently validates the incoming values.
6. If server-side validation fails, the controller returns HTTP 400 with a descriptive message before database work.
7. If validation succeeds, the existing create or update operation proceeds through the existing data model.
8. The existing dashboard display, Edit / Delete targeting, authentication, authorization, and password hashing continue to operate as before.

## 2.5 Error and Data Handling

- Frontend validation failures are shown in the form without sending an API request.
- Invalid direct API requests return HTTP 400 Bad Request and a JSON message; for example: 

```
{"success":false,"message":"Email must follow a valid email format."}
```

.
- Input trimming occurs before length and format checks so surrounding whitespace cannot satisfy or evade validation rules.
- Valid Add and Edit operations continue to use the existing CRUD behaviour and persistence layer.

# 3. Testing & Verification

## 3.1 Testing Approach

Testing covers client-side field validation, negative and boundary cases, server-side validation bypass attempts, valid Add / Edit operations, and regression checks. The supplied implementation report records the listed scenarios as passed. Environment: local development / testing. Browser and API client versions were not specified in the source material.

## 3.2 Test Cases

| Test ID | Test Case                              | Expected Result                                                       | Actual Result                 | Status |
|---------|----------------------------------------|-----------------------------------------------------------------------|-------------------------------|--------|
| TC-01   | Blank / whitespace Registration Number | Reject blank value and show validation error.                         | Submission blocked.           | Pass   |
| TC-02   | Empty First Name                       | Reject the missing required field.                                    | Validation error shown.       | Pass   |
| TC-03   | First Name shorter than 3 chars        | Reject Jo; minimum length is 3.                                       | Validation error shown.       | Pass   |
| TC-04   | First Name contains digits             | Reject John123; alphabetic characters only.                           | Validation error shown.       | Pass   |
| TC-05   | Last Name shorter than 3 chars         | Reject Li; minimum length is 3.                                       | Validation error shown.       | Pass   |
| TC-06   | Invalid Email format                   | Reject john.doe.com.                                                  | Validation error shown.       | Pass   |
| TC-07   | Phone number shorter than 10 digits    | Reject 123456789.                                                     | Validation error shown.       | Pass   |
| TC-08   | Phone number longer than 10 digits     | Reject 12345678901.                                                   | Validation error shown.       | Pass   |
| TC-09   | Phone contains letters                 | Reject 1234567abc; digits only.                                       | Validation error shown.       | Pass   |
| TC-10   | Weak new-student password              | Reject password missing required complexity, e.g. password.           | Validation error shown.       | Pass   |
| TC-11   | Valid student creation                 | Valid input proceeds to the existing API and creates a student.       | Creation succeeds.            | Pass   |
| TC-12   | Valid student update                   | Valid edited data passes validation and updates the intended student. | Update succeeds.              | Pass   |
| TC-13   | Invalid student creation               | Invalid values are rejected and no student record is created.         | Invalid Add request rejected. | Pass   |

| Test ID | Test Case                                     | Expected Result                                                                   | Actual Result                             | Status |
|---------|-----------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------|--------|
| TC-14   | Invalid student update                        | Invalid edited values are rejected and existing data is not updated.              | Invalid Edit request rejected.            | Pass   |
| TC-15   | Backend request with invalid name / Email     | Direct client-bypass request returns HTTP 400 for invalid name or email.          | 400 Bad Request with descriptive message. | Pass   |
| TC-16   | Backend request with invalid Phone / Password | Direct client-bypass request returns HTTP 400 for invalid phone or weak password. | 400 Bad Request.                          | Pass   |
| TC-17   | Existing Edit and Delete targeting            | Existing actions target the intended <code>student_id</code> .                    | Manual regression verified.               | Pass   |
| TC-18   | Administrator authentication / authorization  | Existing access controls continue to operate.                                     | Security regression verified.             | Pass   |

### 3.3 Test Summary

| Test Result         | Count  |
|---------------------|--------|
| Total Test Cases    | 18     |
| Passed              | 18     |
| Failed              | 0      |
| Blocked             | 0      |
| Pass Rate           | 100%   |
| Overall Test Result | Passed |

## 4. Acceptance Criteria Verification

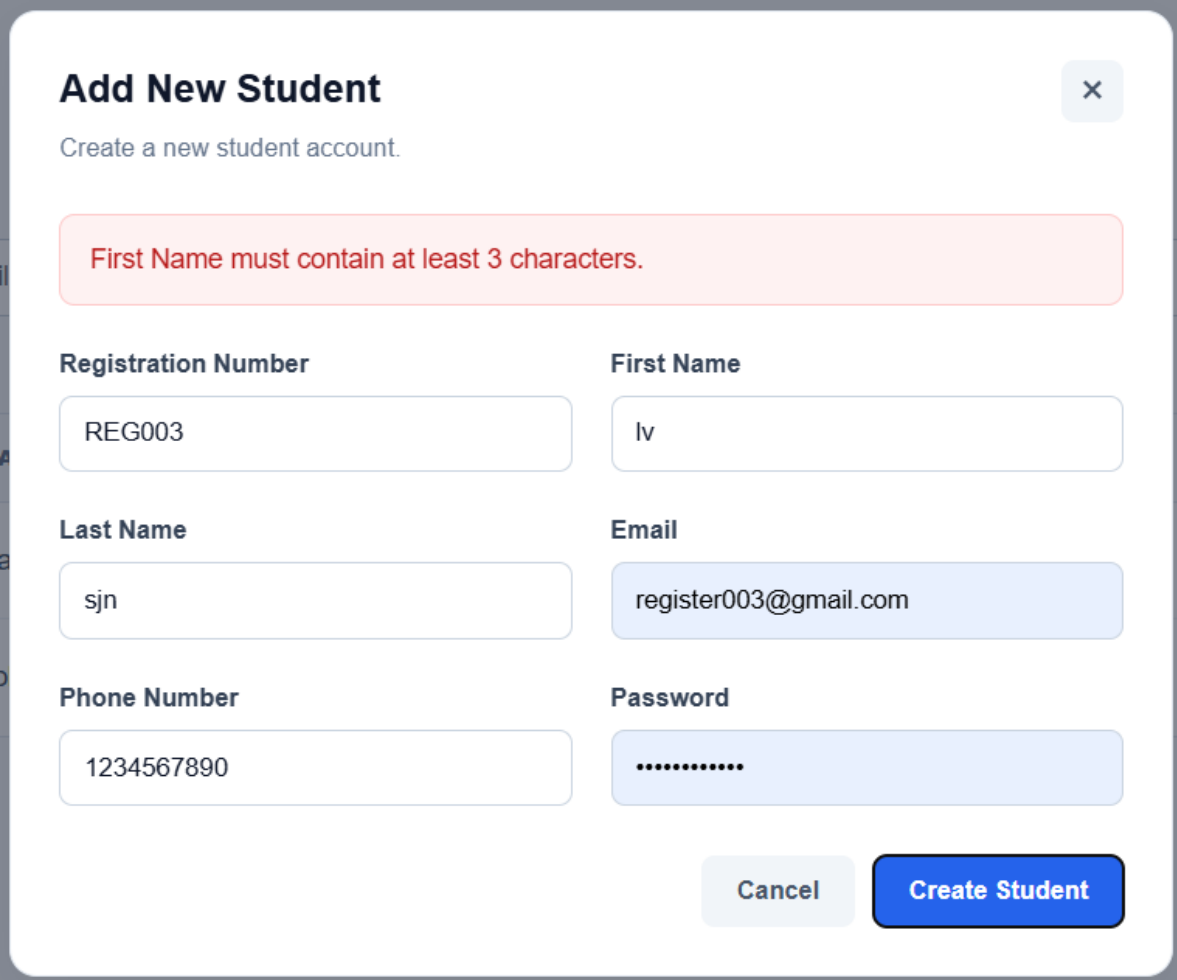
| ID     | Acceptance Criterion                                                            | Verification                           | Status |
|--------|---------------------------------------------------------------------------------|----------------------------------------|--------|
| AC-001 | Registration Number is required and cannot be blank / whitespace.               | Client / server input checks; TC-01    | Met    |
| AC-002 | First Name must be provided.                                                    | Required-field validation; TC-02       | Met    |
| AC-003 | First Name must be at least 3 characters and alphabetic only.                   | UI validation; TC-03, TC-04            | Met    |
| AC-004 | Last Name must be provided and at least 3 characters long.                      | UI validation; TC-05                   | Met    |
| AC-005 | Last Name must contain alphabetic characters only.                              | Backend name validation; TC-15         | Met    |
| AC-006 | Email must match the configured valid email format.                             | UI / API tests; TC-06, TC-15           | Met    |
| AC-007 | Phone Number must contain numeric characters only.                              | Boundary / type tests; TC-09, TC-16    | Met    |
| AC-008 | Phone Number must be exactly 10 digits long.                                    | Boundary tests; TC-07, TC-08, TC-16    | Met    |
| AC-009 | New-student password must be provided and at least 8 characters.                | Password validation; TC-10, TC-16      | Met    |
| AC-010 | New-student password must include upper, lower, number, and special characters. | Password complexity test; TC-10, TC-16 | Met    |
| AC-011 | String inputs are trimmed before validation.                                    | Client / backend logic review          | Met    |
| AC-012 | Valid student creation succeeds.                                                | Creation test; TC-11                   | Met    |
| AC-013 | Valid student update succeeds.                                                  | Update test; TC-12                     | Met    |
| AC-014 | Invalid student creation is rejected.                                           | Negative Add test; TC-13               | Met    |
| AC-015 | Invalid student update is rejected.                                             | Negative Edit test; TC-14              | Met    |
| AC-016 | Direct API requests with invalid data are rejected by the backend.              | HTTP 400 bypass tests; TC-15, TC-16    | Met    |
| AC-017 | Edit and Delete continue to target the correct <code>student_id</code> .        | Manual regression; TC-17               | Met    |
| AC-018 | Administrator authentication and authorization remain functional.               | Security regression; TC-18             | Met    |

## 5. Evidence

Replace each screenshot placeholder with the captured evidence specified for CR-004. Keep the figure caption and criteria mapping to maintain traceability.

### Evidence 01 - Validation Error

Related: input validation criteria / TC-03



The screenshot displays a modal window titled "Add New Student" with a close button (X) in the top right corner. Below the title is the instruction "Create a new student account." A prominent red error message box at the top states: "First Name must contain at least 3 characters." The form contains six input fields arranged in two columns. The left column includes "Registration Number" (containing "REG003"), "Last Name" (containing "sjn"), and "Phone Number" (containing "1234567890"). The right column includes "First Name" (containing "lv"), "Email" (containing "register003@gmail.com"), and "Password" (displayed as dots). At the bottom right, there are two buttons: a light blue "Cancel" button and a dark blue "Create Student" button.

| Field Label         | Field Value           |
|---------------------|-----------------------|
| Registration Number | REG003                |
| First Name          | lv                    |
| Last Name           | sjn                   |
| Email               | register003@gmail.com |
| Phone Number        | 1234567890            |
| Password            | .....                 |

Figure 1: StudentFormModal displaying "First Name must contain at least 3 characters."

## Evidence 02 - Valid Student Creation

Related: AC-012 / TC-11

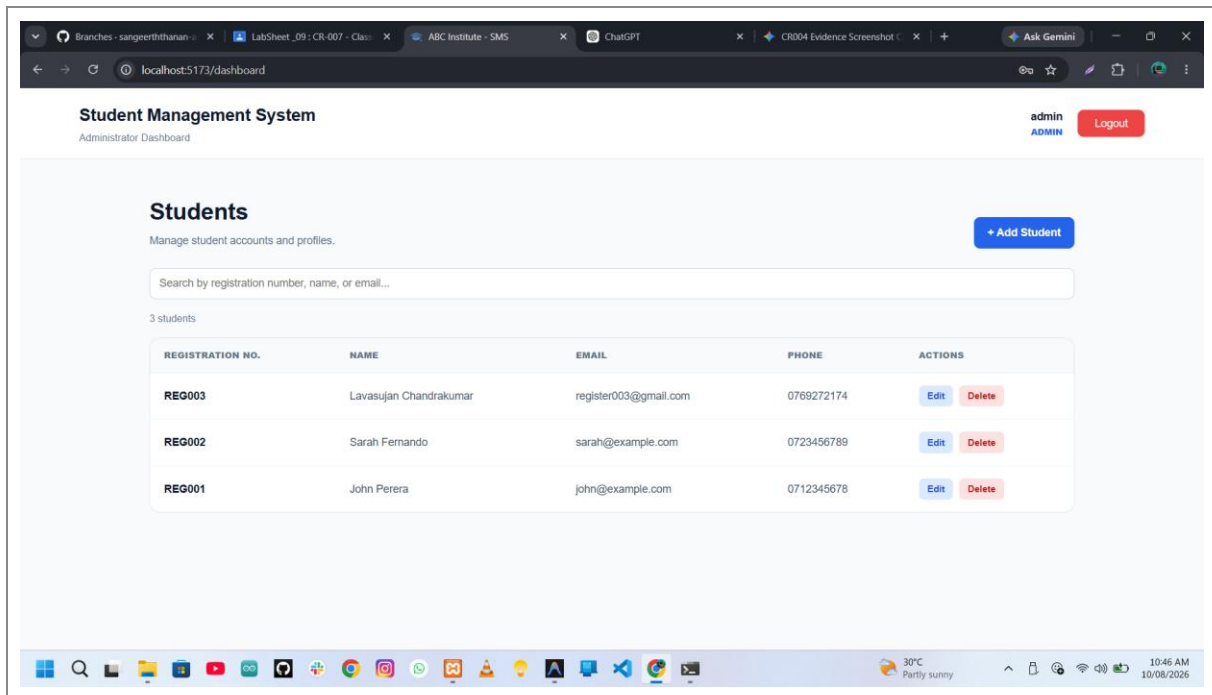


Figure 2: Successful Add Student submission followed by the new record in the dashboard table.

## Evidence 03 - Valid Student Update

Related: AC-013 / TC-12

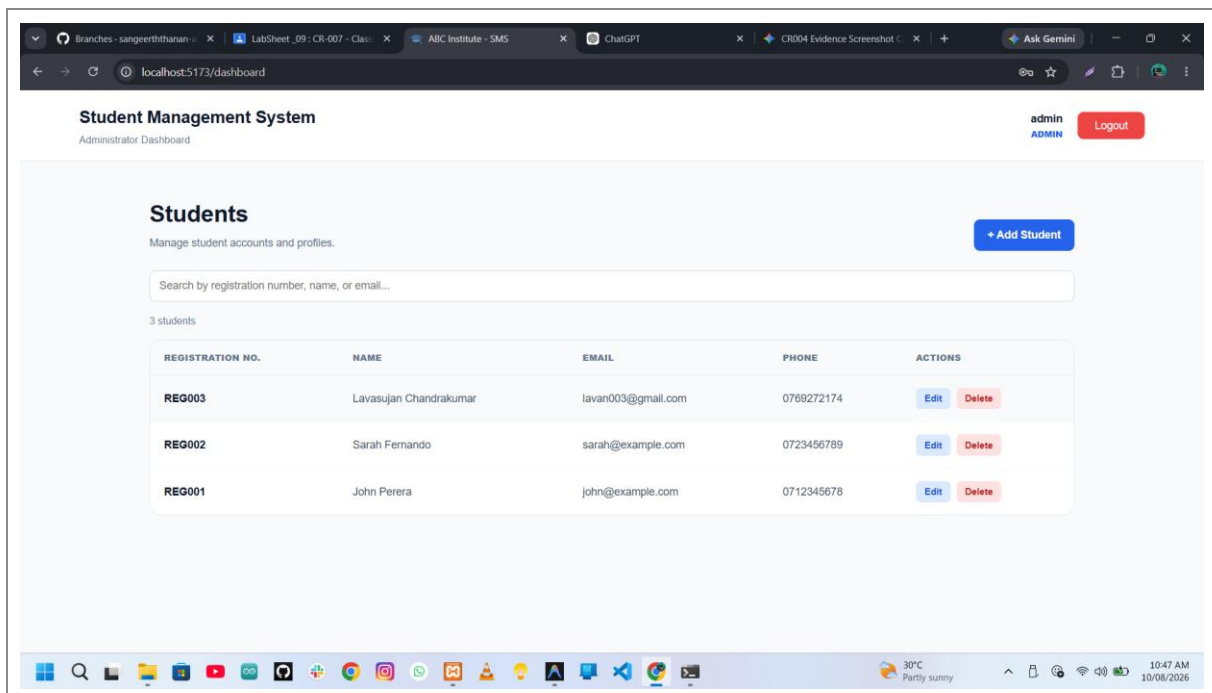


Figure 3: Successful Edit submission with updated student data reflected in the dashboard list.

## Evidence 04 - Backend Rejection

Related: AC-016 / TC-15, TC-16

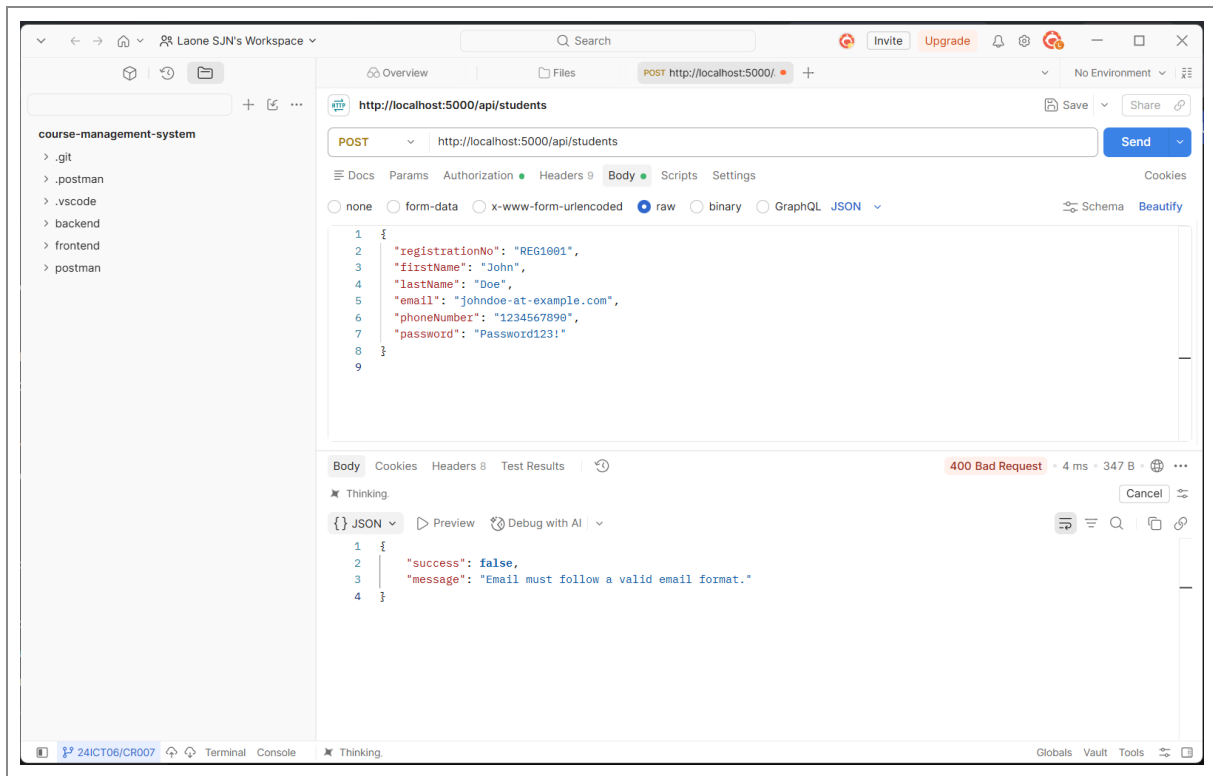


Figure 4: API client or response log showing HTTP 400 for an invalid student request.

## Evidence 05 - Existing Edit and Delete

Related: AC-017 / TC-17

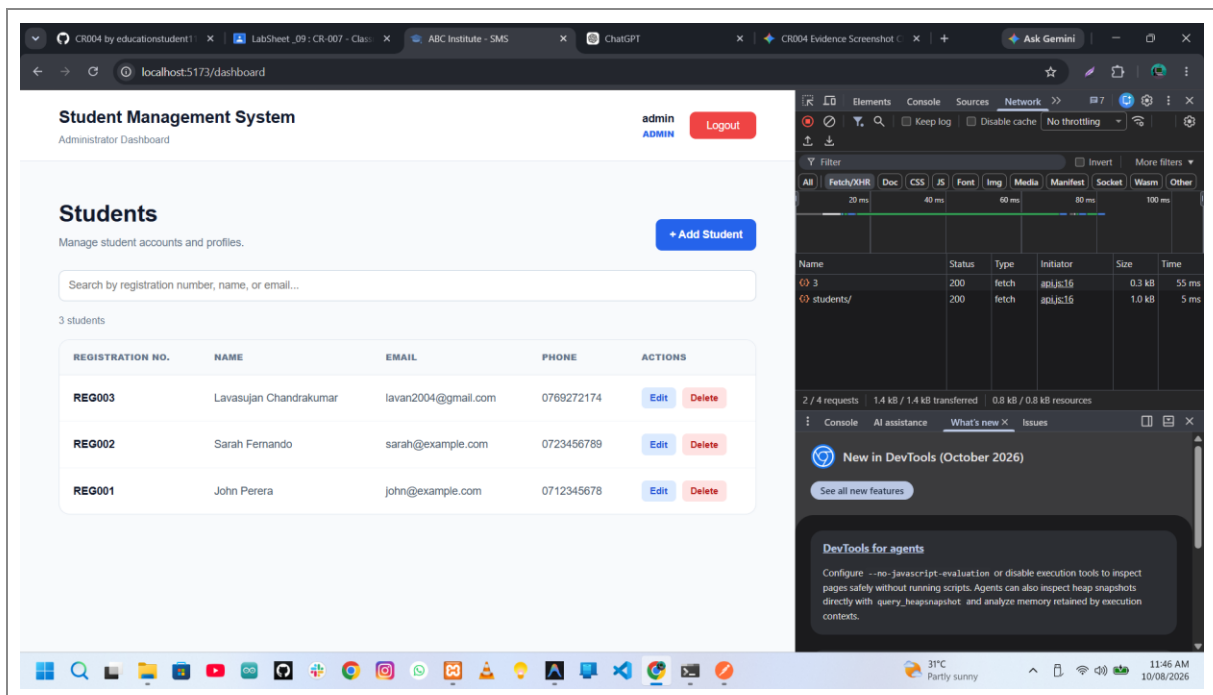


Figure 5: Manual regression evidence showing Edit and Delete target the correct `student_id`.

## 6. Final Result

| Item                 | Result                                                 |
|----------------------|--------------------------------------------------------|
| Requirements         | Met                                                    |
| Implementation       | Completed                                              |
| Testing              | Passed - 18/18 test cases                              |
| Acceptance Criteria  | All 18 criteria met / documented                       |
| Existing CRUD / Auth | Preserved and regression verified                      |
| Final Status         | Implementation complete - screenshot insertion pending |

### 6.1 Final Summary

CR-004 implements client-side and server-side validation for student Add and Edit operations. `StudentFormModal.jsx` provides immediate and specific feedback, while `studentController.js` independently validates and trims request values and rejects invalid direct requests with HTTP 400 before database processing. Name, email, phone, registration number, and new-student password rules are applied, while the existing data model, CRUD, authentication, authorization, and password hashing are preserved. The supplied implementation record reports successful validation, API, and regression checks.

### 6.2 Issues / Limitations

No functional implementation issue is identified in the supplied report. The five screenshot evidence items need to be inserted before submission because the screenshot image files were not included with the Markdown source.

Student Name: C. Lavasujan

Student ID: 24ICT06

Date: 09 October 2026

Signature: \_\_\_\_\_

**~ END OF REPORT ~**